

Pre-verification report – CMTAT

April 2026

About this document. Sequent is a formal verification company that proves systems behave as specified. As part of our preparatory verification process we read and assess a codebase to understand its intended behavior, invariants, and trust boundaries before building a formal model. In the course of that work bugs frequently surface. This *pre_report* communicates those findings to the interested party. It is not a full audit and does not claim completeness.

Reverting snapshot engine halts all state-changing operations

SEVERITY **MEDIUM**

The snapshot administration role is effectively over-privileged. By configuring a reverting snapshot engine either maliciously or by accident, a `SNAPSHOTTER_ROLE` holder can disrupt all state-changing token operations, including transfers, minting, burning, and forced transfers.

This is a role-boundary violation because the snapshot role gains hidden pause-like influence over token liveness. The impact is amplified by an ERC-1404 inconsistency: pre-transfer checks indicate no restrictions even though the actual transfer reverts during snapshot processing.

This can arise from operator error or buggy engine integration and does not require a malicious actor.

A holder of `SNAPSHOTTER_ROLE` can point the token at any engine implementation.

SnapshotEngineModule.sol:L62-L67

```
62 function setSnapshotEngine(  
63     ISnapshotEngine snapshotEngine_  
64 ) public virtual override(ISnapshotEngineModule) onlySnapshotter {  
65     _setSnapshotEngine($, snapshotEngine_);  
66 }
```

The `operateOnTransfer` call sits inside `_update`; any revert there bricks the entire state-changing path.

0_CMTATBaseCommon.sol:L128-L145

```
128 function _update(address from, address to, uint256 amount)  
129     internal virtual override(ERC20Upgradeable)  
130 {  
131     ISnapshotEngine snapshotEngineLocal = snapshotEngine();  
132  
133     if (address(snapshotEngineLocal) != address(0)) {
```

```

134         ERC20Upgradeable._update(from, to, amount);
135         snapshotEngineLocal.operateOnTransfer(
136             from, to, fromBalanceBefore, toBalanceBefore, totalSupplyBefore
137         );
138     } else {
139         ERC20Upgradeable._update(from, to, amount);
140     }
141 }

```

Snapshot-engine failures are not modelled in the ERC-1404 preflight, so `detectTransferRestriction` can return `TRANSFER_OK` while the real transfer reverts.

ValidationModuleERC1404.sol:L104-L110 and L147-L150

```

104 uint8 codeReturn = _detectTransferRestriction(from, to, value);
105 if (codeReturn != uint8(IERC1404Extend.REJECTED_CODE_BASE.TRANSFER_OK)) {
106     return codeReturn;
107 } else if (address(ruleEngine_) != address(0)) {
108     return ruleEngine_.detectTransferRestriction(from, to, value);
109 }
110
111 if (deactivated()) {
112     return uint8(...TRANSFER_REJECTED_DEACTIVATED);
113 } else if (paused()) {
114     return uint8(...TRANSFER_REJECTED_PAUSED);
115 }

```

setFrozenTokens can exceed balance, breaking core accounting invariant

SEVERITY **MEDIUM**

The `setFrozenTokens` function can violate the token’s core accounting invariant by allowing an account’s frozen amount to exceed its real balance. Once that invalid state exists, the token can no longer reason correctly about “active” balance, so ordinary transfers, forced transfers, balance helpers, and some compliance checks can start reverting unexpectedly.

This does not require a malicious actor. An administrator or an `ERC20ENFORCER_ROLE` can trigger it accidentally through bad input, buggy tooling, or an incorrect operational script.

The issue also affects the token’s internal mint path. Because minting is modeled as a transfer from `address(0)`, assigning frozen tokens to `address(0)` can break mint operations globally because of the same issue.

`_setFrozenTokens` writes the new value with no balance check — frozen can be set above the real balance.

ERC20EnforcementModuleInternal.sol:L33-L45

```

33 function _setFrozenTokens(address account, uint256 value)
34     internal virtual returns (bool)
35 {
36     uint256 frozenTokensLocal = $_frozenTokens[account];
37     if (value > frozenTokensLocal) {
38         $_frozenTokens[account] = value;
39         _freezeTokensEmitEvents(account, difference, value, "");
40     }
41     return true;
42 }

```

Contrast: `_freezePartialTokens` correctly preserves `frozen <= balance` with a `require`.

ERC20EnforcementModuleInternal.sol:L52-L60

```

52 function _freezePartialTokens(address account, uint256 value, bytes memory data
53     )
54     internal virtual
55 {
56     uint256 balance = ERC20Upgradeable.balanceOf(account);
57     uint256 frozenTokensLocal = $_frozenTokens[account] + value;
58     require(balance >= frozenTokensLocal,
59         CMTAT_ERC20EnforcementModule_ValueExceedsAvailableBalance());
60     $_frozenTokens[account] = frozenTokensLocal;
61 }

```

Downstream: `activeBalance` assumes `frozenTokens <= balance`, and mint is modelled as a transfer from `address(0)` — so a frozen balance on `address(0)` breaks minting globally.

ERC20EnforcementModuleInternal.sol:L139-L144 and 0_CMTATBaseCommon.sol:L159-L160

```

139 function _checkActiveBalance(address from, uint256 value)
140     internal virtual view returns (bool isValid, uint256 activeBalance)
141 {
142     uint256 frozenTokensLocal = _getFrozenTokens(from);
143     activeBalance = ERC20Upgradeable.balanceOf(from) - frozenTokensLocal;
144 }
145
146 function _mintOverride(address account, uint256 value) internal virtual
147     override {
148     _checkTransferred(address(0), address(0), account, value);
149 }

```

***crosschainMint, crosschainBurn and burnFrom* bypass RuleEngine restrictions**

SEVERITY **MEDIUM**

CMTAT's RuleEngine design suggests that issuers can enforce operator-specific compliance rules, distinguishing direct holder transfers from third-party actions such as delegated burns or bridge burns. In `burnFrom`, `crosschainMint`, and `crosschainBurn`, however, the real operator identity is lost before the RuleEngine hook is invoked, causing the token to use the generic transfer path instead of the spender-aware path.

As a result, an operator that still holds `BURNER_FROM_ROLE` or `CROSS_CHAIN_ROLE` that would be rejected under the ERC-7551-style `canTransferFrom` and ERC-1404 `detectTransferRestrictionFrom` can still burn tokens through `burnFrom`, and a bridge that should fail operator-based compliance can still execute `crosschainBurn` and `crosschainMint`. In the `crosschainMint` case, this is particularly serious because a blocked bridge can still mint new tokens and inflate supply while RuleEngine says that the operator should not be permitted to act.

This is a compliance-boundary failure, rather than a true access-control bypass. The token exposes spender-aware compliance semantics, but those semantics do not consistently apply to burn/mint-style operator flows. The issue does not require a malicious actor. It can arise whenever an issuer relies on the RuleEngine to disable or constrain a previously authorized operator, redemption agent, or bridge while the corresponding roles remain active. In that state, the issuer may reasonably expect policy enforcement to block the operator, but burn operations can still succeed.

NOTE ON PRIOR AUDIT

A previous audit by Nethermind Audit Agent (*CMTAT_AuditAgent_Report_Comment_v3.1.0.pdf*) raised a superficially similar concern, but framed it more broadly as a generic claim that mint, burn, and minter-initiated transfers should preserve spender identity for RuleEngine checks. That finding was rejected on the basis that regular transfer, mint, and burn flows do not have a spender, and that only `transferFrom` does.

The rejection does not apply to the issue described here. This finding is limited to `burnFrom`, `crosschainBurn`, and `crosschainMint`, which are explicitly operator-initiated flows: `burnFrom` uses `msg.sender` as the allowance spender, while `crosschainBurn` and `crosschainMint` are executed by a bridge operator subject to `CROSS_CHAIN_ROLE`. In these cases, the protocol already recognizes a concrete acting operator, but that identity is discarded before the RuleEngine write hook is invoked, causing spender-aware restrictions to be bypassed.

Entry points: the bridge operator (`onlyTokenBridge`) and the `BURNER_FROM_ROLE` caller (`onlyBurnerFrom`) are both known to the protocol at this point.

ERC20CrossChainModule.sol:L60-L87

```

60 function crosschainMint(address to, uint256 value)
61     public virtual override(IERC7802) onlyTokenBridge
62 {
63     _mintOverride(to, value);
64     emit CrosschainMint(to, value, _msgSender());
65 }
66
```

```

67  function burnFrom(address account, uint256 value)
68      public virtual override(IBurnFromERC20) onlyBurnerFrom
69  {
70      address sender = _msgSender();
71      _burnFrom(sender, account, value);
72  }

```

But `_mintOverride` and `_burnOverride` call `_checkTransferred` with `address(0)` as the spender — the operator identity is thrown away before the RuleEngine is reached.

0_CMTATBaseCommon.sol:L159-L170

```

159  function _mintOverride(address account, uint256 value)
160      internal virtual override(ERC20MintModuleInternal)
161  {
162      _checkTransferred(address(0), address(0), account, value);
163  }
164
165  function _burnOverride(address account, uint256 value)
166      internal virtual override(ERC20BurnModuleInternal)
167  {
168      _checkTransferred(address(0), account, address(0), value);
169  }

```

Consequence: when `spender == address(0)`, the write hook falls back to the spender-less RuleEngine path, skipping any operator-specific rules.

ValidationModuleRuleEngine.sol:L137-L145

```

137  function _transferred(address spender, address from, address to, uint256 value)
138      internal virtual
139  {
140      if (spender != address(0)) {
141          ruleEngine_.transferred(spender, from, to, value);
142      } else {
143          ruleEngine_.transferred(from, to, value);
144      }
145  }

```

Asymmetry: the read-only preflight still exposes the spender-aware entry point, so off-chain checks and on-chain execution disagree about whether the operator is allowed.

ValidationModuleERC1404.sol:L114-L128

```

114  function detectTransferRestrictionFrom(
115      address spender,
116      address from,
117      address to,

```

```

118     uint256 value
119 ) public virtual view override(IERC1404Extend) returns (uint8 code) {
120     return ruleEngine_.detectTransferRestrictionFrom(spender, from, to, value);
121 }
```